

Deep Hedging

Mathieu Paradis, Frederic Godin

Department of Mathematics and Statistics, Concordia University

20.07.2026

*Note: This document is heavily inspired by the two following papers; all credit is to the authors:

Carbonneau, A. and Godin, F. *Equal Risk Pricing of Derivatives with Deep Hedging*

Buehler, H., Gonon, L., Teichmann, J., & Wood, B. (2019). *Deep Hedging*.

Table of Contents

Introduction

Market Set Up

Types of Algorithms

Equal Risk Pricing

Advantages and Disadvantages

Code Implementation

Hedging problems in incomplete markets

In practice, markets are deemed incomplete. Typical hedging strategies, like Black-Scholes, fail to hedge perfectly under these conditions and can depend on subjective assumptions.

The approach we take is through **Deep Hedging**, using **Reinforcement Learning** and **Neural Networks** to take into account the market environment without the need for Greeks to optimize our hedging strategy.

Curse of Dimensionality

Definition: The **curse of dimensionality** is a phenomenon that arises when the characteristics of data become more complex as dimensionality increases.

It often leads to a **decrease in the performance** of statistical learning models that use local information. The amount of data required also **increases exponentially**.¹

Deep learning algorithms often overcome the curse of dimensionality seen in many real-world financial models, in part because it can **adapt to the structure of the data**.

¹D. Peng, Z. Gui, and H. Wu, “Interpreting the Curse of Dimensionality from Distance Concentration and Manifold Effect,” arXiv preprint, 2024.

Notation - Probability Space and Assets

All asset prices, rewards, and hedging strategies are defined over the **probability space** $(\Omega, \mathcal{F}, \mathbb{P})$.

- ▶ Ω represents every possible future market evolution
- ▶ $\mathcal{F}$ the set of observable events
- ▶ and $\mathbb{P}$ the real-world probability measure

The financial market has a **discrete finite time horizon** of T years, with known and fixed **trading dates**

$$\mathcal{T} := \{0 = t_0 < t_1 < \dots < t_N = T\}.$$

There are $D + 1$ liquid, tradable **assets**:

- ▶ D risky assets
- ▶ 1 risk-free asset

Notation - Prices of assets

Let the non-negative price process of the **D risky assets** be denoted by $\{S_n\}_{n=0}^N$, where $S_n := [S_n^{(1)}, \dots, S_n^{(D)}]$ are the prices at $t_n \in \mathcal{T}$

Let the process of the **risk-free asset** be denoted by $\{B_n\}_{n=0}^N$, with $B_n := \exp(rt_n)$, where $r \in \mathbb{R}$ is the annualized continuously compounded **risk-free rate**.

For simplicity, we assume no asset pays any cash flows in $[t_0, t_{N-1}]$.

Notation - Market Filtration

We define the **market filtration** $\mathbb{F} := \{\mathcal{F}_n\}_{n=0}^N$, where $\mathcal{F}_n$ represents the **information available** at time n .

The filtration is generated by the **asset price process** $\mathcal{F}_n := \sigma(S_u \mid u = 0, \dots, n)$, meaning all available information at time n is obtained from observed price history. Therefore this implies $\mathcal{F}_s \subseteq \mathcal{F}_t$ for $s < t$.

We also assume $\mathcal{F} = \mathcal{F}_N$, implying that at time N we have the **complete market history**.

Notation - Market information

Let $\{Z_n\}_{n=0}^N$ be a $\mathbb{F}$ -adapted process vector, with Z_n a K -dimensional. The $\{Z_n\}_{n=0}^N$ process includes state variables relevant to the **dynamics** of the risky assets and the pricing of contingent claims.

For example, a **European-type** contingent claim payoff is denoted $\Phi(S_N, Z_N)$, with $\Phi : [0, \infty)^D \times \mathbb{R}^K \rightarrow \mathbb{R}$

Notation - Hedging strategy

Let $\delta := \{\delta_n\}_{n=0}^N$ be a **trading strategy** to minimize risk exposure, where $\delta_n := [\delta_n^{(0)}, \delta_n^{(1)}, \dots, \delta_n^{(D)}]$ with $n = 1, \dots, N$ is a vector that contains the **amount of shares held in each risky asset** in the period $(t_{n-1}, t_n]$ in the hedging portfolio.

Other assumptions:

- ▶ The initial portfolio is only invested in the risk-free asset
- ▶ All assets and contingent claims are assumed well-behaved and integrable enough
- ▶ Trading the risky assets is assumed not to impact their prices.

Discounted gain process

Let $\{G_n^\delta\}_{n=0}^N$ be the **discounted gain process** of strategy δ , where at time t_n before rebalancing we have:

$$G_n^\delta := \sum_{k=1}^n \delta_k^{(1:D)} \cdot (B_k^{-1} S_k - B_{k-1}^{-1} S_{k-1}), \quad n = 1, 2, \dots, N, \quad (1)$$

Market Frictions: Transaction Costs

Let $c_k(x)$ be the **cost function** representing the penalty of trading x shares at time t_k .

The total cost of a strategy δ up to maturity N is:

$$C_T(\delta) := \sum_{k=0}^n c_k(\delta_k - \delta_{k-1}) \quad (2)$$

An admissible cost function must be non-negative, normalized ($c_k(0) = 0$) and upper semi-continuous.

Choosing a Cost function

A possible choice of cost function is **proportional transaction cost**.

For a portfolio rebalance from δ_n to δ_{n+1} , the proportional transaction cost applied to the risky assets with parameter κ is:

$$c_n \left(\delta_{n+1}^{(1:D)} - \delta_n^{(1:D)} \right) := \sum_{d=1}^D c_k^d S_n^{(d)} \left| \delta_{n+1}^{(d)} - \delta_n^{(d)} \right| \quad (3)$$

Other cost functions include:

- ▶ Fixed transaction costs
- ▶ Complex cross-asset costs

Discounted Gain Process with Frictions

Because transaction costs deplete the risk-free asset balance, the discounted gain process G_N^δ must be explicitly reduced by the cumulative sum of all **trading fees** and other **market frictions** incurred over the life of the strategy.

Market frictions are included in the cost function for convenience.

At terminal time t_N , the gain process becomes:

$$G_N^\delta := \sum_{k=1}^N \delta_k^{(1:D)} \cdot (B_k^{-1} S_k - B_{k-1}^{-1} S_{k-1}) - \sum_{k=0}^{N-1} B_k^{-1} c_k \left(\delta_{k+1}^{(1:D)} - \delta_k^{(1:D)} \right) \quad (4)$$

Self-financing

The process δ is a **self-financing strategy** if it is *predictable* and if the *rebalancing costs are absorbed by the cash account*:

$$\delta_{n+1}^{(1:D)} \cdot S_n + \delta_{n+1}^{(0)} B_n + c_n \left(\delta_{n+1}^{(1:D)} - \delta_n^{(1:D)} \right) = \delta_n^{(1:D)} \cdot S_n + \delta_n^{(0)} B_n \quad (5)$$

for $n = 0, 1, \dots, N - 1$.

As the Deep Hedging strategy is often assumed to be **self-financing**, the only change in portfolio value is from **profits captured in the gains process**. This is equivalent to:

$$\delta_n^{(0)} B_n = B_n \left(V_0 + G_n^\delta \right) - \delta_n^{(1:D)} \cdot S_n \quad (6)$$

where G_n^δ implicitly incorporates all accumulated transaction costs.

Hedging Portfolio Value

Let $\{V_n^\delta\}_{n=0}^N$ as the hedging portfolio **value process** under strategy δ where:

$$V_n^\delta := \delta_n^{(1:D)} \cdot S_n + \delta_n^{(0)} B_n \quad n = 0, 1, \dots, N. \quad (7)$$

It can also be shown that:

δ is self-financing $\iff V_n^\delta = B_n(V_0 + G_n^\delta)$ for $n = 0, 1, \dots, N$.

Risk Measure

We chose ρ to be a **convex risk measure** mapping a random loss variable to a real number. We also want to choose a risk measure that **does not penalize gains**.

A popular choice is to use the **root semi-quadratic penalty (RSQP)**.

Let $R = -\mathcal{P}_\delta$ be the negative total profits at time T under hedging strategy δ , then:

$$\rho^{RSQP}(R) = \sqrt{E[R^2 \mathbb{I}_{\{R>0\}}]} \quad (8)$$

Other possible popular convex risk measures include:

- ▶ Conditional Value at Risk (CVaR)
- ▶ Entropic Risk

The Optimisation Problem

The optimal risk (or pricing) of a derivative $\Phi(S_N, Z_N)$ becomes a **minimization over all predictable strategies Π** :

$$\pi(\Phi) = \min_{\delta \in \Pi} \rho \left(\Phi(S_N, Z_N) - \underbrace{\sum_{k=1}^N \delta_k^{(1:D)} \Delta \tilde{S}_k}_{\text{Frictionless Gains}} + \underbrace{\sum_{k=0}^{N-1} B_k^{-1} c_k (\Delta \delta_k)}_{\text{Frictions \& Penalties}} \right) \quad (9)$$

Simplifying the terms inside the brackets forms our total discounted gain process G_N^δ :

$$\pi(\Phi) = \min_{\delta \in \Pi} \rho \left(\Phi(S_N, Z_N) - G_N^\delta \right) \quad (10)$$

Neural Network Engine

To solve the optimization problem under Deep Hedging, we use a **neural network** parameterized by weights θ .

Deep Hedging typically uses a set of **Feed-Forward Neural Networks** (*FFNN*)

Let $X \in \mathbb{R}^{d_{in} \times 1}$ be a feature vector of dimensions $d_{in} \in \mathbb{N}$ and $L, d_1, \dots, d_{L-1}, d_{out} \in \mathbb{N}$ with $L \geq 2$

Define the FFNN as the mapping $F_\theta(X) : \mathbb{R}^{d_{in}} \rightarrow \mathbb{R}^{d_{out}}$ with the set of trainable parameter $\theta := \{W^{(1)}, \dots, W^{(L)}, b^{(1)}, \dots, b^{(L)}\}$

²

²Carbonneau, A. and Godin, F. *Equal Risk Pricing of Derivatives with Deep Hedging*.

Neural Network Engine

We have

$$F_{\theta}(X) := A_L \circ F_{L-1} \circ \dots \circ F_1,$$

$$F_l := \sigma \circ A_l, \quad l = 1, \dots, L-1, \quad (11)$$

- ▶ $\circ$ being the composition operator
- ▶ σ be the activation function $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ that is applied element-wise to outputs
- ▶ $A_l(Y) := W^{(l)}Y + b^{(l)}$ for any $l = 1, \dots, L$

Where:

$$l = 1 : \quad W^{(1)} \in \mathbb{R}^{d_1 \times d_{\text{in}}}, \quad b^{(1)} \in \mathbb{R}^{d_1 \times 1}, \quad Y \in \mathbb{R}^{d_{\text{in}} \times 1},$$

$$2 \leq l \leq L-1 : \quad W^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}, \quad b^{(l)} \in \mathbb{R}^{d_l \times 1}, \quad Y \in \mathbb{R}^{d_{l-1} \times 1},$$

$$l = L : \quad W^{(L)} \in \mathbb{R}^{d_{\text{out}} \times d_{L-1}}, \quad b^{(L)} \in \mathbb{R}^{d_{\text{out}} \times 1}, \quad Y \in \mathbb{R}^{d_{L-1} \times 1}. \quad (12)$$

³Carbonneau, A. and Godin, F. *Equal Risk Pricing of Derivatives with Deep Hedging*.

Sets of Neural Network Engine

Let $\mathcal{NN}_{\infty, d_{in}, d_{out}}^{\sigma}$ be the **set of all FFNN mappings** from $\mathbb{R}^{d_{in}} \rightarrow \mathbb{R}^{d_{out}}$ with fixed σ and arbitrary number of neurons.

Let $\Theta_{M, d_{in}, d_{out}} \subseteq \mathbb{R}^{R_M}$, where $M \in \mathbb{N}$, $R_M \in \mathbb{N}$, define

$$\mathcal{NN}_{M, d_{in}, d_{out}} := \{F_{\theta} : \theta \in \Theta_{M, d_{in}, d_{out}}\}. \quad (13)$$

As the set of Neural Network as defined in (11) with $\theta \in \Theta_{M, d_{in}, d_{out}}$.

Definition for sets of Neural Networks

There are 2 possible definitions for $\mathcal{NN}_{M,d_{\text{in}},d_{\text{out}}}$, with interchangeable notations:

- ▶ The set of FFNNs has a **fixed structure** $L^{(M)}$, and neurons per layer. This implies θ is a fixed set of hyperparameters.
- ▶ The set of FFNNs has an **arbitrary number of layers and neurons per layer**. This is useful under the complete optimization problem of neural networks, called **hyperparameter tuning**.

4

⁴Carbonneau, A. and Godin, F. *Equal Risk Pricing of Derivatives with Deep Hedging*.

Optimal hedging with Neural Networks

Since trade decisions are generated by the **k-th neural network** F_{θ_k} on market features X_{k-1} so that $\delta_k^{(1:D)} = F_{\theta_k}(X_{k-1})$. We then have the **cost function** $J(\theta)$ to be minimized.

$$J(\theta) = \min_{\delta \in \Pi} \rho\left(\Phi(S_N, Z_N) - G_N^{\delta(\theta)}\right) \quad (14)$$

Optimal hedging with Neural Networks

Under standard **stochastic gradient descent** and $\theta_o \in \Theta_{M_S}$ be the initial parameter value of F_θ , we update the trainable parameters iteratively.

$$\theta_{j+1} = \theta_j - \eta_j \nabla_\theta J(\theta_j) \quad (15)$$

Let ∇_θ be the gradient and η_j a small positive deterministic value that typically reduces as j increases that represents the learning rate.

Evaluating the gradient

We then use a **minibatch** to estimate the gradient function. Let $N_{batch} \in \mathbb{N}$ and the simulated minibatch $\mathbb{B} := \{\pi_{i,j}\}_{i=1}^{N_{batch}}$ with

$$\pi_{i,j} := \Phi(S_{N,i}, Z_{N,i}) - B_N G_{N,i}^{\theta(S, \theta_j)}. \quad (16)$$

being the i th hedging error under the parameters $\theta = \theta_j$.

We estimate the gradient cost function $\nabla_{\theta} J(\theta_j)$ with $\nabla_{\theta} \hat{\rho}(\mathbb{B}_j)$ evaluated at $\theta = \theta_j$, where $\hat{\rho}(\mathbb{B}_j)$ is the empirical estimator of $\rho(\pi_{i,j})$

A popular analytical approach to compute the gradient of a cost function is to use **backpropagation**. Coding libraries such as TensorFlow are often used for deep learning for backpropagation.

Gaussian regime switching

The daily log returns $y_n := \log(S_n/S_{n-1})$ are assumed to follow

$$y_{n+1} = \mu_{h_n} \Delta + \sigma_{h_n} \sqrt{\Delta} \epsilon_{n+1}, \quad n = 0, \dots, N-1 \quad (17)$$

They are also assumed to follow a **Gaussian regime-switching model**. Let $h = \{h_n\}_{n=0}^N$ be a finite Markov chain, where h_n is the state of the market during the period $[t_n, t_{n+1})$, with transition probabilities $\{\gamma_{i,j}\}_{i=1,j=1}^{H,H}$ and $n = 0, \dots, N-1$.

Define $\xi := \{\xi_n\}_{n=0}^N$ as the **predictive process** with $\xi_n := [\xi_{n,1}, \dots, \xi_{n,H}]$. We have that $\xi_{n,j}$ is the probability that the Markov chain is in the j th regime during $[t_n, t_{n+1})$

$$\xi_{n,j} := \mathbb{P}(h_n = j \mid \mathcal{F}_n), \quad j = 1, \dots, H. \quad (18)$$

Markov chain regime

We see that the **feature vector** to be considered for the neural network is $X_n = [S_n, V_n, \xi_n, T - t]$, which depends on the current price, current portfolio value, current market regime, and current time.

Using the Gaussian pdf and setting ξ_0 as the **stationary distribution of the Markov chain**, we can recursively compute with $n = 1, \dots, N$:

$$\xi_{n,i} = \frac{\sum_{j=1}^H \gamma_{ji} \phi_j(y_n) \xi_{n-1,j}}{\sum_{j=1}^H \phi_j(y_n) \xi_{n-1,j}}, \quad i = 1, \dots, H. \quad (19)$$

Steps of Deep Hedging

The training loop for Deep Hedging translates into a computational algorithm. For each training iteration j :

1. **Market Simulation (Data Generation):** Simulate a minibatch of N_{batch} paths for the asset prices S_n and state variables Z_n under the physical probability measure $\mathbb{P}$.
2. **Forward Pass (Trading Execution):** For each path and time step t_k , feed the market features X_{k-1} into the neural network to output the holding positions: $\delta_k^{(1:D)} = F_{\theta_j}(X_{k-1})$.
3. **Loss Evaluation (Risk Measurement):** Compute the terminal discounted gains $G_{N,i}^{\delta(\theta)}$ (including transaction costs) and the terminal hedging error $\pi_{i,j}$ for each path. Aggregate these to calculate the empirical risk loss $J(\theta_j)$.
4. **Backward Pass (Parameter Update):** Use backpropagation to compute the gradient $\nabla_{\theta} \hat{\rho}(\mathbb{B}_j)$. Update the network weights using gradient descent: $\theta_{j+1} = \theta_j - \eta_j \nabla_{\theta} J(\theta_j)$.

Then, repeat steps until the cost function $J(\theta)$ converges to a min. 27 / 55

Comparison of Reinforcement learning Types

As of now, we have used a **direct policy optimization** approach. The hedging strategy is represented by a neural network whose parameters are optimized using stochastic gradient descent.

Other reinforcement learning approaches can be used to train the neural network. These approaches differ in how the policy is learned and optimized:

- ▶ **Value-based**: Learn the value of different actions.
- ▶ **Policy-based**: Directly learn the optimal policy.
- ▶ **Actor-critic**: Combines value and policy based methods.

Reinforcement Learning Algorithms

Algorithm	Type	Action Space	Hyperparameters
DQL	Value-based	Discrete	17
Double DQL	Value-based	Discrete	19
Dueling DQL	Value-based	Discrete	17
DD DQL	Value-based	Discrete	19
MCPG	Policy-based	Continuous	9
PPO	Actor-critic	Continuous	17
DDPG	Actor-critic	Continuous	22
TD3	Actor-critic	Continuous	27

Table 1: Caption

DQL

Deep Q-Learning (DQL) is a widely used discrete DRL algorithm that derives the optimal policy from an action-value function derived from the Bellman equation. The agent selects actions based on the estimated value of each possible action.

The optimal **action-value function** is the expected value of the rewards when taking:

- ▶ action a
- ▶ in state s
- ▶ under optimal policy π^*
- ▶ with discounting factor γ

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_t = s, a_t = a, (a_{u+1})_{u=t}^{\infty} \sim \pi^* \right] \quad (20)$$

DQL

Under the DQL method, the **optimal action-value function** is represented by a neural network with parameter set $\phi = \{\phi\}_{i=0}^{\mathcal{I}-1}$, estimated iteratively over $\mathcal{I}$ iterations.

DQL then follows

$$S_t \rightarrow FFNN \rightarrow Q^\pi(s, a) \rightarrow \underset{a}{\operatorname{argmax}} \rightarrow a_t \quad (21)$$

DQL is then an attractive approach if the action space is **discrete** and if you want to compare the **value of actions**.

In our case, since we have **continuous actions** δ , DQL might not be the best fit.

Monte Carlo Policy Gradient

Monte Carlo Policy Gradient is a policy-based method with lower computational complexity than value-based as we increase dimension.

We simulate multiple **hedging losses** R and use its distribution to assess the value of a given policy.

The policy parameters are updated iteratively under **stochastic gradient descent** (15) with batches of $R = \{R\}_{n=1}^N$ of N i.i.d. loss variables simulated under the policy parameter θ_i , learning rate α , and risk measure $\rho(X)$.

Equal Risk Pricing

One application of Deep Hedging is to price derivatives. A recognized approach is through **equal risk pricing**, setting the option price as the value such that the global risk exposure for a long and a short is equal under optimal hedging strategies

Equal risk pricing

Let $\epsilon^{(L)}(V_0)$ and $\epsilon^{(S)}(V_0)$ be respectively be the long and short **risk exposure** under the optimal hedge. We have:

$$\epsilon^{(L)}(V_0) := \min_{\delta \in \mathcal{I}} \rho(-\Phi(S_N, Z_N) - B_N(V_0 + G_N^\delta)), \quad (22)$$

$$\epsilon^{(S)}(V_0) := \min_{\delta \in \mathcal{I}} \rho(\Phi(S_N, Z_N) - B_N(V_0 + G_N^\delta)) \quad (23)$$

And as a direct consequence of the translation invariance of ρ ,

$$\epsilon^{(L)}(V_0) = \epsilon^{(L)}(0) - B_N V_0 \quad (24)$$

$$\epsilon^{(S)}(V_0) = \epsilon^{(S)}(0) - B_N V_0 \quad (25)$$

Equal risk pricing

Let δ^L and δ^S be respectively the **optimal long and short hedging strategies** for a derivative:

$$\delta^{(L)} := \arg \min_{\delta \in \Pi} \rho \left(-\Phi(S_N, Z_N) - B_N(V_0 + G_N^\delta) \right), \quad (26)$$

$$\delta^{(S)} := \arg \min_{\delta \in \Pi} \rho \left(\Phi(S_N, Z_N) - B_N(V_0 + G_N^\delta) \right). \quad (27)$$

Through translation invariance of ρ , δ^L and δ^S do not depend on V_0 :

$$\delta^{(L)} := \arg \min_{\delta \in \Pi} \rho \left(-\Phi(S_N, Z_N) - B_N G_N^\delta \right), \quad (28)$$

$$\delta^{(S)} := \arg \min_{\delta \in \Pi} \rho \left(\Phi(S_N, Z_N) - B_N G_N^\delta \right). \quad (29)$$

Equal risk pricing

The **equal risk price**, denoted C_0^* of the contingent claim is defined as the initial portfolio value such that the measured risk exposure of the optimally hedged long and short are equal.

$$\epsilon^{(L)}(-C_0) = \epsilon^S(C_0) \quad (30)$$

Assuming there exists a finite super-replication price for the contingent claim. The equal risk price **exists, is unique and arbitrage-free** and can be expressed as:

$$C_0^* = \frac{\epsilon^{(S)}(0) - \epsilon^{(L)}(0)}{2B_N} \quad (31)$$

Let ϵ^* be the residual level of risk faced by the hedger.

$$\epsilon^* := \epsilon^{(L)}(-C_0) = \epsilon^S(C_0) \quad (32)$$

Advantages of Deep Hedging

Some of the advantages were previously discussed in this presentation, for example:

- ▶ Overcomes the curse of dimensionality
- ▶ Flexibility in risk preferences/ risk measures
- ▶ Computationally efficient
- ▶ Optimisation across the total time horizon
- ▶ Adaptable to real market dynamics

Advantages of Deep Hedging

In general, Deep hedging is particularly great to compute costs and assessing liquidity at scale since

- ▶ **TensorFlow**: is a fairly straightforward implementation tool
- ▶ **Asset class agnostic**: it can handle most asset classes
- ▶ **Parallelizable**: computationally more efficient, as it allows for splitting the mathematic problem into smaller tasks
- ▶ **Speed even when scaled**: It can handle high-dimensional portfolios, as once the neural network is trained we simply need a large matrix multiplication.
- ▶ **Handles more complex contingent claims**

Advantages of Deep Hedging

A major advantage of Deep Hedging is that we do not need to use any **Greeks** (used in other popular hedging approaches). Using Greeks is not always optimal because of the following drawbacks:

- ▶ **Local optimization:** primarily focuses on local sensitivities, while DH learns a hedging strategy over the entire horizon.
- ▶ **Difficult to incorporate transaction costs:** DH incorporates costs directly in its discounted gains process.
- ▶ **Ignores volatility skew:** The standard Black-Scholes Greeks may require additional model-based estimates of implied volatility and its dynamics, while DH adapts to them.
- ▶ **Requires Mark-to-Model:** Under the Greeks approach, we must select a model and calibrate it, while under DH, it learns the optimal strategy from market dynamics

Disadvantages and limitations

Using Neural networks to optimize the hedging strategies also comes with some drawbacks:

- ▶ **Synthetic data bias:** as real historical data is often too sparse to train deep networks, most hedging papers rely on synthetic data, which may not be able to reflect rare events.
- ▶ **Black box effect:** Like some neural network, for some reinforcement learning algorithms, we have limited visibility into what happens during training, hence can create regulatory problems. It also does not provide any interpretable rationale for its recommended holding strategy.
- ▶ **Potentially inefficient trades:** As the strategy dynamically updates, small offsetting trades can be triggered to adjust the position, which can increase unnecessary transaction costs. Some no-transaction boundaries can be implemented to reduce not necessary trading, but they are not built-in by default.

Disadvantages and limitations

- ▶ **Simulation to real markets:** finding reliable estimates used to generate training data can be challenging or computationally expensive.
- ▶ **Computational overhead:** training neural networks is very computationally expensive and requires more computational power (better GPUs) than other standard closed-form methods.

Some of these drawbacks can be partially mitigated using some interesting methods, but such methods are out of scope.

Code Implementation - Step 1

The following slides explain how to **implement Deep Hedging** under simplified assumptions. The code is mostly based on the paper Carbonneau, A. and Godin, F. *Equal Risk Pricing of Derivatives with Deep Hedging*

See this GitHub repository as an example:  Deep Hedging Onboarding

Step 1: Import the necessary python libraries

- ▶ **Tensorflow or Pytorch** → for machine learning
- ▶ **numPy** → for Mathematical functions and arrays
- ▶ **matplotlib** → for Plotting
- ▶ **yfinance** → for historical market data

Code Implementation - Step 2

Step 2: Set parameters for the training and testing set using simulated data

- ▶ $[\mu, \sigma]$ of the underlying using yahoo finance data
- ▶ option type (Call or Put)
- ▶ Strike price (for ATM, Strike = current price)
- ▶ Maturity and number of hedging steps
- ▶ Risk-free rate
- ▶ Number of paths for the training and test sets
- ▶ Proportional cost parameter
- ▶ Stock price preprocessing choice (log-moneyness)

Code Implementation - Step 3

Step 3: Simulate the training environment. As real markets **do not have rich enough data** to train a neural network we simulate price paths $\{S_n\}_{n=0}^N$ under GBM dynamics:

$$S_n = S_0 \exp \left[\sum_{j=1}^n \left(\mu - \frac{\sigma^2}{2} \right) d + \sigma \sqrt{d} Z_j \right], \quad Z_j \sim \mathcal{N}(0, 1). \quad (33)$$

Then, we transform the prices into log-moneyness $X = \log \left(\frac{S}{K} \right)$ before entering the network, which **improves training stability** whilst keeping a consistent price scale.

Finally, split the simulated data set into **training** and **test** sets.

Code Implementation - Step 4

Step 4: Set the **hyperparameters** for the Neural Network using either optimization problems, or fixed hyperparameters.

For simplicity, we use fixed hyperparameters, using the same values as those used in Carbonneau, A., & Godin, F. (2021).

The hyperparameters are the following:

- ▶ **Batch size:** Number of samples processed before updating the model parameters.
- ▶ **Epochs:** Number of complete passes through the dataset.
- ▶ **Hidden layers:** Number of intermediate neural network layer.
- ▶ **Number of Neurons:** Number of Neurons per hidden layers
- ▶ **Learning rate:** Step size controlling how much the model parameters are adjusted during training.
- ▶ **Level of significance:** Threshold for the risk measure.

Code Implementation - Step 5

Step 5: Compute the discount factors to discount future cashflows to present values, using the risk-free rate.

This computes $B_n^{-1} = e^{-rt_n}$ at each time step.

Code Implementation - Step 6 - Reorder

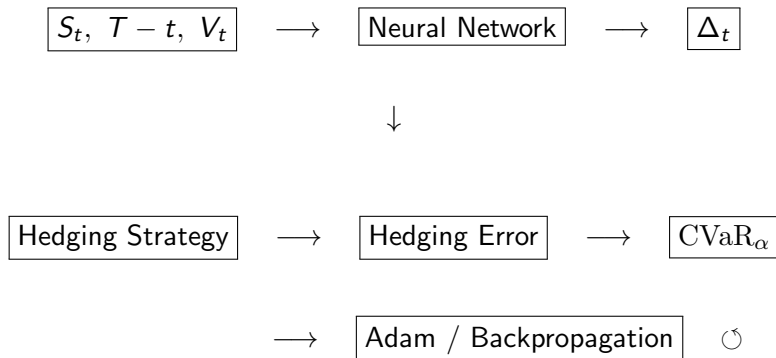
Step 6: Set up the algorithm to implement the **Feed Forward Neural Network** with weights θ as defined earlier.

1. Define the parameters for the deep hedging model
2. Build the Network with hidden layers and ReLU activation function, outputting the hedging position in the underlying
3. Go through every hedging time step
 - ▶ Forward propagate and store the outputs at time t (hedging positions)
 - ▶ Implement proportional transaction cost to account for market frictions
 - ▶ Update the portfolio value, which becomes part of the next NN decision

Code Implementation - Step 6

5. Calculate the total net discounted trading gains
6. Calculate the CVaR from the hedging error. The selected risk measure represents the loss function to optimize
7. Use backpropagation to compute the gradients of the loss with respect to θ
8. Update the network with weights θ using Adam optimizer
9. Train using mini-batches over multiple epochs
10. Use the learned network parameters to predict the holding ratio of the test set.

Deep Hedging: Feed-Forward Neural Network



Code Implementation - Step 7/8

Step 7: Evaluate the **measured risk** under the selected risk measure on a held-out dataset to show an **out-of-sample risk evaluation**.

Step 8: Train the long and short positions for the designated underlying under the chosen option type to find the optimal stock position at time zero after the given number of epochs with the following feature vector $X_t = [S_t, T - t, V_t]$.

For simplicity, we do not take into account regime switching as inputs.

Code Implementation - Step 9

Step 9: Find the **equal risk price**, the price that equalizes residual hedging risk between the long and the short sides at time zero, as in (31).

Also find the **residual hedging risk** of the option as the average between the residual hedging risk (32) of the long and the short.

We can also show the development of the **ideal stock position** over time until maturity, showing when rebalancing is necessary.

Conclusion

We have seen that **Deep Hedging** offers a good alternative to traditional methods as it adapts to the dynamics of the underlying rather than relying on a closed-form solution pricing model.

Deep hedging uses **neural networks** to directly learn hedging strategies by minimizing risk exposure.

Transaction costs, liquidity constraints and other market dynamics **can be incorporated** into the optimization problem.

Overall, Deep hedging offers a **flexible alternative** to traditional hedging methods, since markets are incomplete.

References I

Buehler, H., Gonon, L., Teichmann, J., & Wood, B. (2019).
Deep Hedging.
Quantitative Finance, 19(8), 1271–1291.

Carbonneau, A., & Godin, F. (2021).
Equal Risk Pricing of Derivatives with Deep Hedging.
Quantitative Finance, 21(4), 593–608.

Neagu, A., Godin, F., & Kosseim, L. (2025).
Deep Reinforcement Learning Algorithms for Option Hedging.
arXiv preprint arXiv:2504.05521.

Carbonneau, A. (2021).
Pricing and Hedging Financial Derivatives with Reinforcement Learning Methods.
PhD thesis, Concordia University.

Ruf, J., & Wang, W. (2020).
Neural Networks for Option Pricing and Hedging: A Literature Review.
Journal of Computational Finance, 24(1), 1–46.

References II

Lütkebohmert, E., Schmidt, T., & Sester, J. (2022).
Robust Deep Hedging.
Quantitative Finance, 22(8), 1465–1480.

Imaki, S., Imajo, K., Ito, K., Minami, K., & Nakagawa, K. (2021).
No-Transaction Band Network: A Neural Network Architecture for Efficient Deep Hedging.
arXiv preprint arXiv:2103.01775.

Poddar, M. (2026).
Uncertainty-Aware Deep Hedging.
arXiv preprint arXiv:2603.10137.

Bollerslev, T. (2009).
Glossary to ARCH (GARCH).
Duke University, CREATES, and NBER.
Version dated February 16, 2009.

References III

GeeksforGeeks. (2025).

Adam Optimizer.

GeeksforGeeks.

Published June 19, 2025. Accessed July 22, 2026.

Peng, D., Gui, Z., & Wu, H. (2024).

Interpreting the Curse of Dimensionality from Distance Concentration and Manifold Effect.

arXiv preprint.

Wuhan University, China.